CLIMP: Contrastive Language-Image Mamba Pretraining

A reproduction and critical study

Shabtay, Zimerman, Schwartz, Giryes — arXiv 2601.06891 (2026)

Tanish Wanve · CS613 paper presentation

Code, results and every number on these slides: github.com/tanishwanve-git/climp-cs613



The problem: CLIP's Transformer towers have three bottlenecks

CLIP learns one embedding space for images and text from (image, caption) pairs. It is the backbone of zero-shot classification, image–text retrieval, and the vision side of most multimodal LLMs.

Bottleneck	Concretely	Consequence
Quadratic attention in the ViT image tower	224 px → 14×14 = 196 patches; 896 px → 56×56 = **3,136 patches**. 16× more pixels ⇒ **256× more** attention entries	High resolution is expensive; the learned position grid only fits one size
77-token text limit	Dense captions are long: DOCCI averages **~142 tokens, 94% exceed 77**	Everything after token 77 is silently cut off
Spurious correlations	Pairwise attention can latch onto non-robust cues	Fragile under distribution shift

Why it matters: CLIP is infrastructure. Any weakness in its encoders propagates into every retrieval system and vision-language model built on top of it.

What existing work fixed — and what it left open

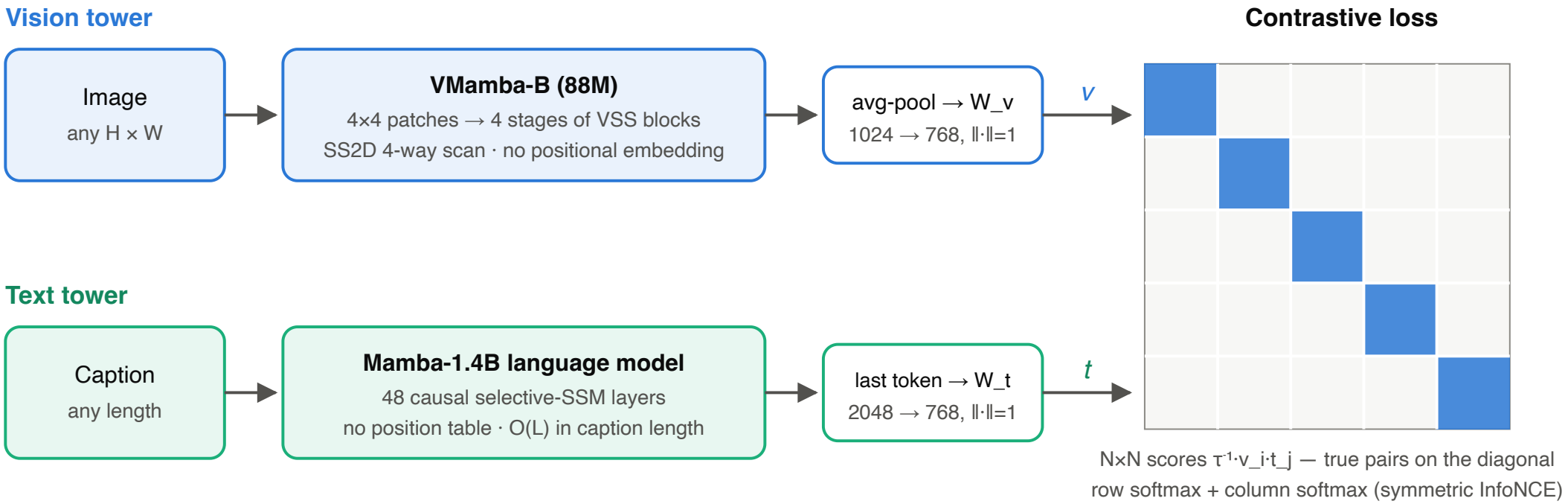
Line of work	What it fixes	What remains
FlexViT, NaFlex, RoPE-ViT	ViT at variable resolution	Still $O(L^2)$; needs special position schemes or extra training
LLM2CLIP, UniViTAR — an LLM as the text tower	Long, rich text	Vision tower is still a Transformer
CLIP-Mamba (Huang et al. 2024)	First Mamba inside CLIP	Hybrid towers only; narrow evaluation
Vim, VMamba — vision SSMs	Linear-time vision, spatial inductive bias	Never tested as a full CLIP backbone

What motivated the authors

State-space models (Mamba) process sequences in **linear time**, have **no position table**, and carry a **locality bias**. Each of these properties addresses one of the three bottlenecks.

> **Hypothesis:** make *both* CLIP towers SSMs, and the model inherits all three benefits at once.

The core idea: CLIP built entirely from state-space models



CLIP: ViT ($O(L^2)$ attention, fixed patch grid) + Transformer text (77 tokens). **CLIMP:** both towers are SSMs, linear in L, no fixed size or length.

What is new: (1) the first CLIP with an SSM in **both** towers · (2) a **controlled** comparison: every model trained on the same data (CC12M) with the same recipe · (3) a broad evaluation: retrieval, OOD, resolution, long text, geometry, efficiency, inductive bias.

The key insight — and why it should work

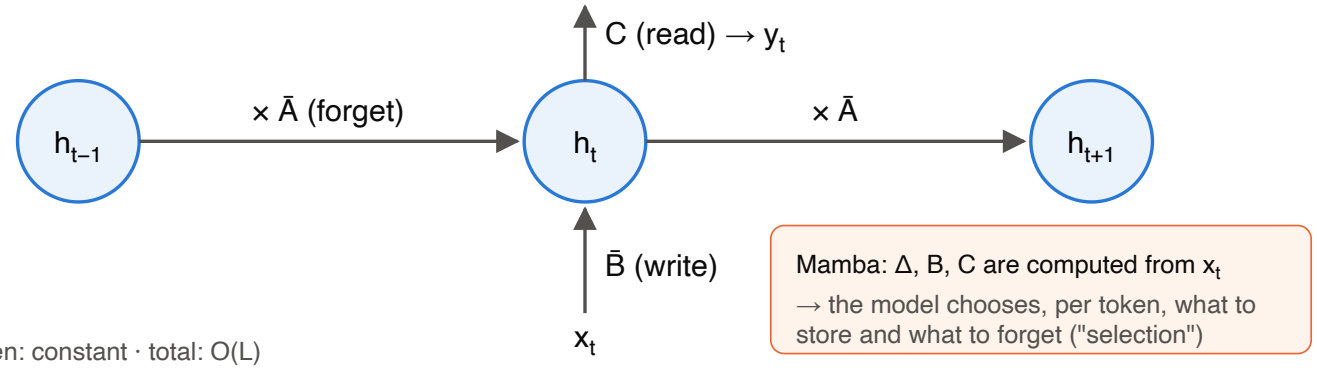
> **In an SSM, the scan order *is* the positional encoding.** Nothing in the model depends on how long the sequence is.

Claim	Mechanism	Paper's evidence
Any resolution, no retraining	No position table; the same weights scan a 14×14 or a 56×56 patch grid	Table 4: retrieval at 224 / 512 / 896 px
Any caption length	Causal LM with no position table; the last token has read everything	Tables 5–6: long-caption retrieval
Robustness, better retrieval	The recurrence decays with distance (Mamba-2: state from k steps back weighted by λ^k) → locality bias; tighter alignment, lower hubness	Tables 2, 7, 8
Efficiency	Linear-time scan instead of quadratic attention	Fig. 3: memory and FLOPs

These four claims are exactly what my experiments test, using the released checkpoints.



How Mamba works: a learned, selective recurrence



cost per token: constant · total: O(L)

Continuous SSM

$$h'(t) = A\,h(t) + B\,x(t), \qquad y(t) = C\,h(t)$$

Discretised with step Δ (zero-order hold)

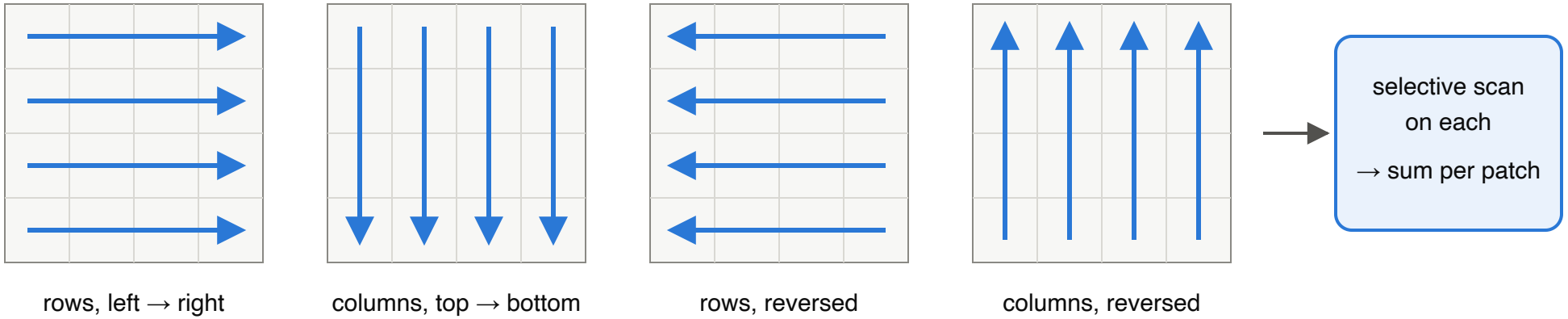
$$\bar{A} = e^{\Delta A}, \quad \bar{B} \approx \Delta B, \quad h_t = \bar{A}\,h_{t-1} + \bar{B}\,x_t$$

Large Δ: reset and focus on this token. Small Δ: keep the past, mostly ignore this token.

	Mamba-1	Mamba-2
Δ, B, C	depend on x_t (**selection**)	depend on x_t
A	per channel	scalar · I (SSD)
Speed	linear scan	**2–8× faster**
CLIMP NoCaps R@1 (ours)	**38.2**	37.4

Vision tower: VMamba's 2-D selective scan (SS2D)

One patch grid, unfolded into four 1-D sequences



After 4 scans every patch has received context from all directions (global receptive field), still O(L) in the number of patches. Position is encoded only by the scan order, so the same weights run on a 14×14 grid (224 px) or a 56×56 grid (896 px).

- An image is not a sequence, so VMamba **unfolds the patch grid four ways**, scans each, and sums the results per patch
- Hierarchical: 4 stages with patch merging (like a CNN), then average-pool → W_v
- CLIMP sets `posembed=False`, so position comes only from the scan order

I checked this in the code. VMamba-B has identical parameter names and shapes when built for 224 px and for 896 px. **One set of weights serves every resolution**: no interpolation, no retraining.

(assertion in exp_nocaps.py; logged in E1.log)

Text tower and training objective

Last-token pooling

$$t_{\text{raw}} = H_k, \qquad k = \max\{\, i : \text{token } i \text{ is not padding} \,\}$$

- Mamba is **causal**: h_t only sees tokens ≤ t, so the **last real token is the only one that has read the whole caption**. This is the same reason GPT-style embedders use the last token.
- With no position table, captions of **any length** are accepted
- Check: right padding (Mamba-1) and left padding (Mamba-2) give identical embeddings: cosine **1.00000**

Symmetric InfoNCE (CLIP's loss)

$$\mathcal{L} = -\frac{1}{2N}\sum_{i=1}^{N}\left[\log\frac{e^{v_i\cdot t_i/\tau}}{\sum_j e^{v_i\cdot t_j/\tau}} + \log\frac{e^{t_i\cdot v_i/\tau}}{\sum_j e^{t_i\cdot v_j/\tau}}\right]$$

- In words: in each **row and column** of the N×N similarity matrix, the true pair must win a softmax
- The other N−1 captions in the batch act as **negatives**, so CLIP needs big batches (paper: 2048)
- τ is learned (initial value 0.07); embeddings are unit-length, so v·t is a cosine

Training procedure and design decisions (paper)

Recipe (identical for all models)

- CC12M, 10 epochs, 224 px
- AdamW, cosine, peak LR 5e-5
- Batch 2048, 768-d joint space
- Init: VMamba-B ← ImageNet-1K; text ← pretrained Mamba-1 1.4B / Mamba-2 1.3B
- Baselines: RoPE-ViT / FlexViT / NaFlex + **LLaMA-3.2-1B** text tower, same recipe

Design decisions

Decision	Why
SSM on **both** sides	Linear cost and no fixed size on either modality
Causal pretrained LM, not a bidirectional SSM	Mature checkpoints exist; Hydra (bidirectional) was worse
No positional embedding	Any-resolution inference for free

Ablation (Table 10), Acc@1 on 28 datasets: VMamba + Mamba-1 **29.6** vs VMamba + LLaMA **26.1**. RoPE-ViT barely changes with the text tower (27.3 → 28.1). The authors' reading: **matched SSM towers learn more compatible representations**.



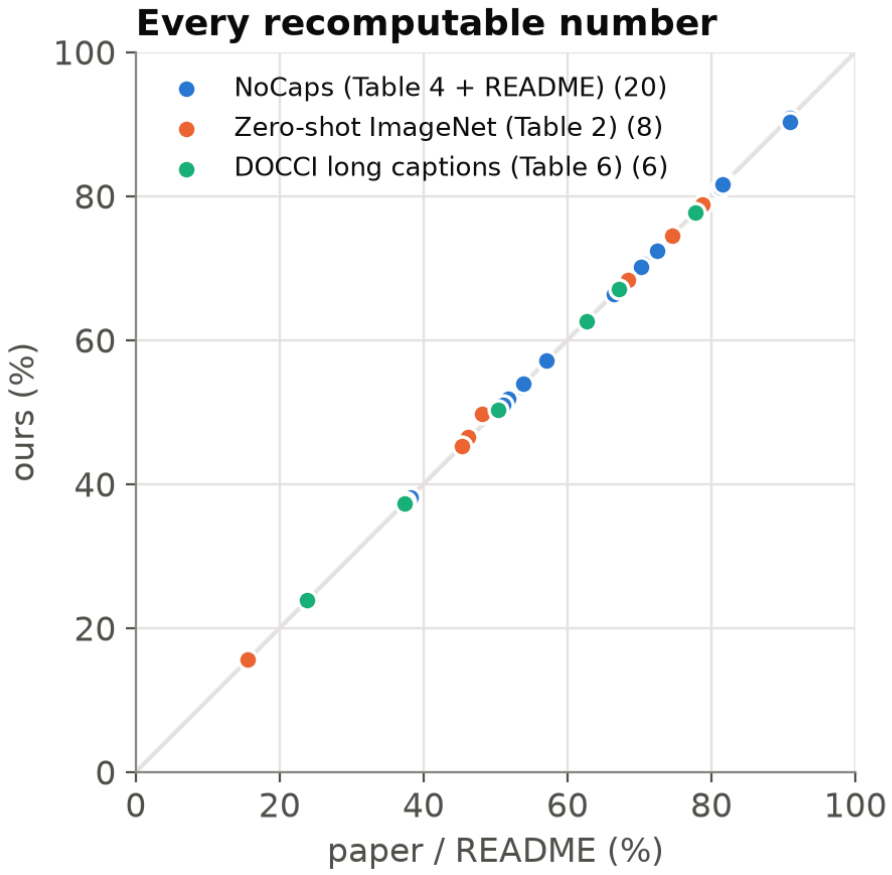
My study: setup

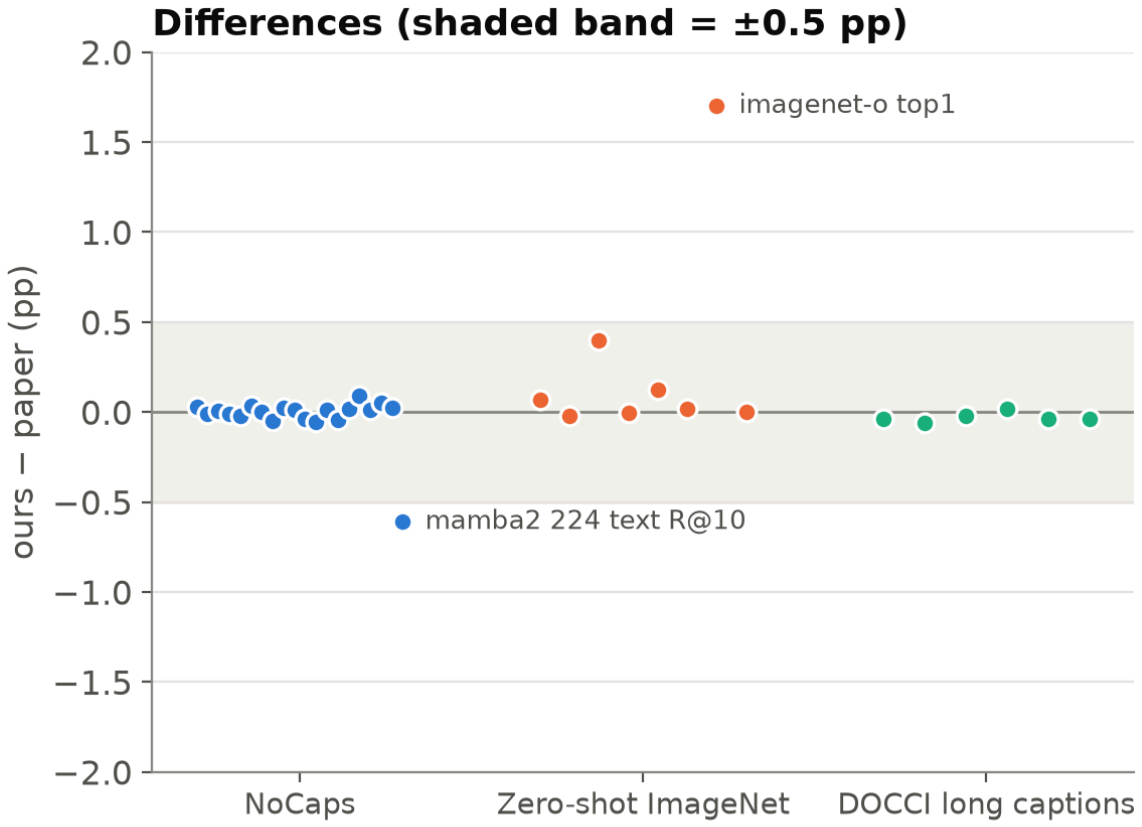
ID	Experiment	Data	Paper
E1	NoCaps retrieval @224/512/896	4,500 img · 45,000 captions	Table 4
E2	Alignment · uniformity · hubness	NoCaps @224	Table 8
E3	Zero-shot ImageNet-V2/R/A/O	10k / 30k / 7.5k / 2k	Table 2
E5	DOCCI long captions @896	5,000 pairs	Table 6
E6	Memory + FLOPs vs resolution	—	Fig. 3
E8	Tiny VMamba vs ViT, CIFAR-10, 3 seeds	50k / 10k	Table 7
NEW	CSLS · text truncation · patch shuffle · SugarCrepe		—

- **Models:** the authors' **released** CLIMP-Mamba1 / Mamba2 checkpoints
- **Baseline: OpenCLIP ViT-B/16 (LAION-2B)**. The paper's baselines are unreleased; same size (86 M vs 88 M); the paper itself cites it. ⚠ **~167× more data**, so *not* controlled
- **Metrics:** R@k, top-1/5, alignment / uniformity / hubness
- **Compute:** Kaggle 2× T4, fp16 (as upstream)
- **Correctness gate:** E1 must match the README within ±0.5 pp first



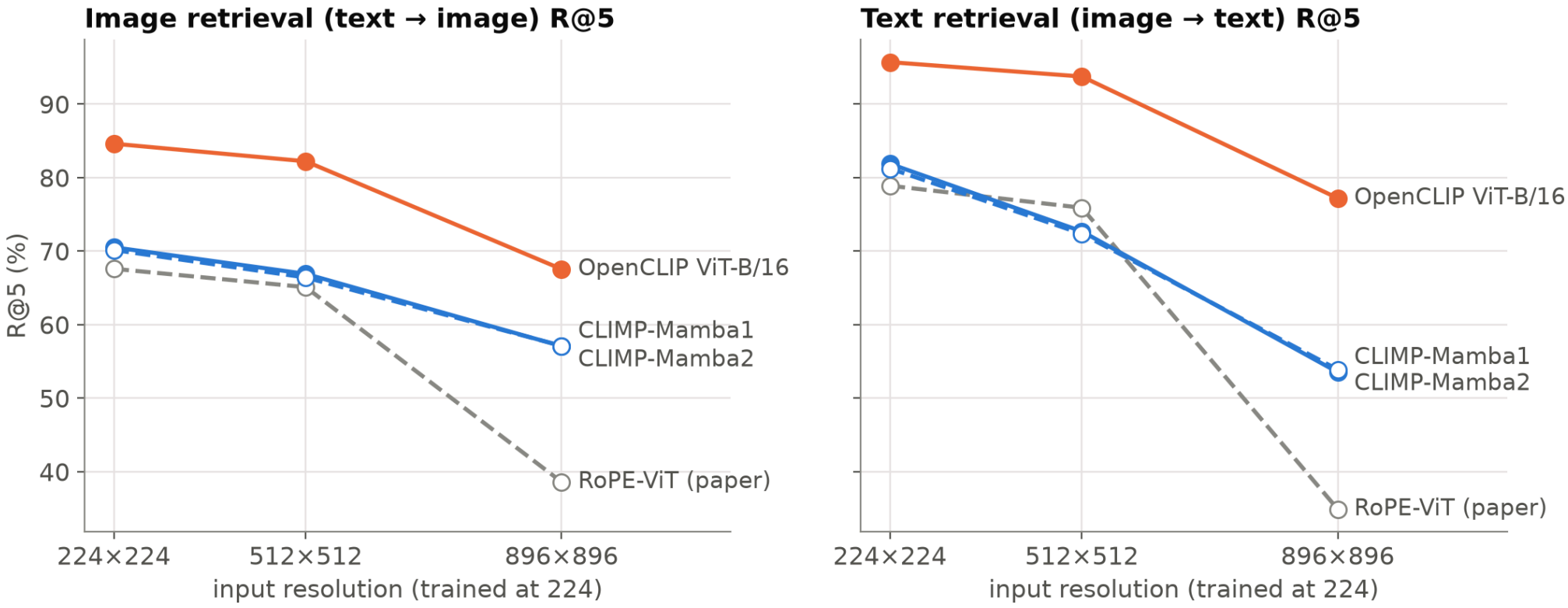
Reproduction at a glance: the released models behave exactly as reported





32 of 34 recomputable numbers match within ±0.5 pp (median gap **0.02 pp**). Outliers: ImageNet-O top-1 (+1.7) and Mamba-2 text R@10 (–0.6). These are the authors' trained weights, so a match validates my *evaluation*; I retrain CLIMP myself later (slides 19–20).

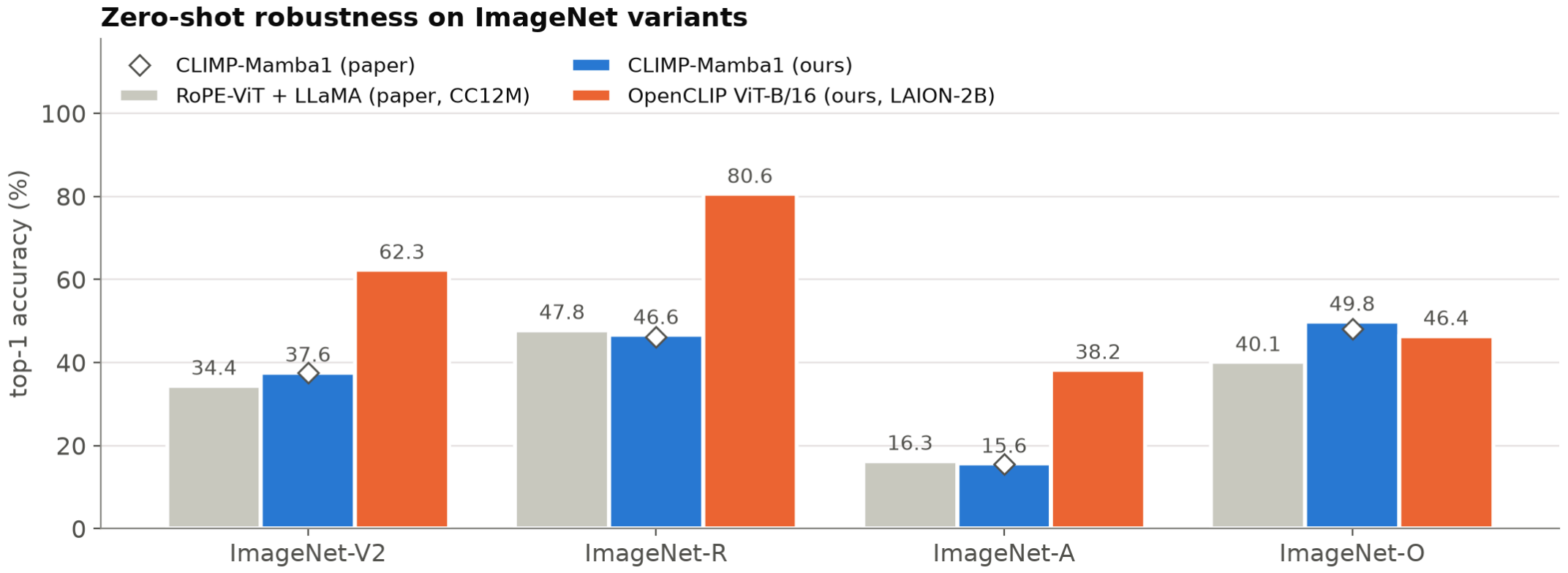
Resolution scaling (E1, Table 4): reproduced, but the comparison matters



REPRODUCED Every cell of Table 4 matches. Against the paper's RoPE-ViT, CLIMP degrades far more gracefully at 896 px (57.1 vs 38.6 image R@5).

NEW A public ViT with plain interpolated positions stays ahead at every resolution. It loses image R@5 at the same rate (–20% vs –19%), while CLIMP's text R@5 drops more (–35% vs –19%).

Zero-shot robustness (E3, Table 2)



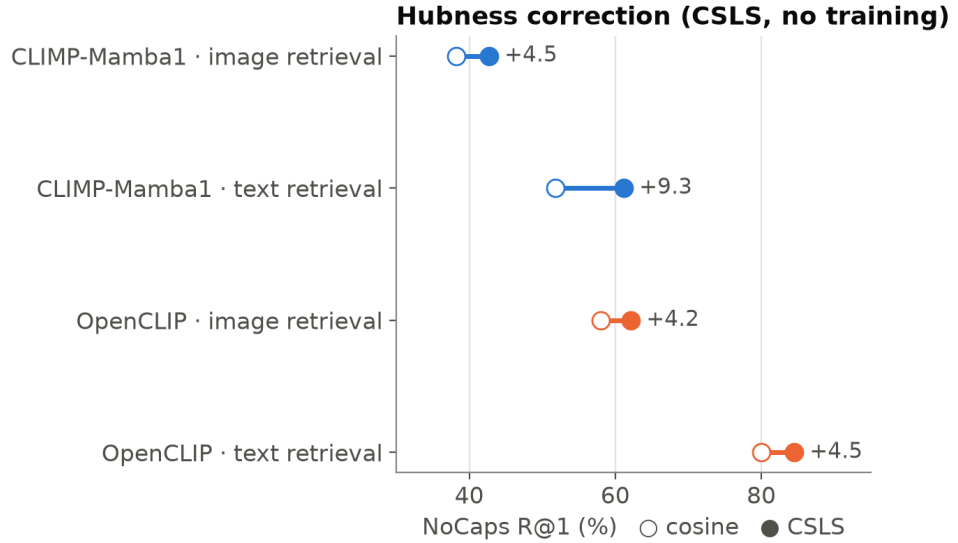
REPRODUCED Top-5 matches exactly; top-1 within 0.4 pp (ImageNet-O +1.7). Against the paper's CC12M baseline, CLIMP wins on V2 and O.

The headline **"beats CLIP trained on 167× more data"** holds on **ImageNet-O only** (49.8 vs 46.4). On V2, R and A, OpenCLIP leads by 23–34 pp.

Embedding geometry (E2, Table 8) and a free fix for hubness

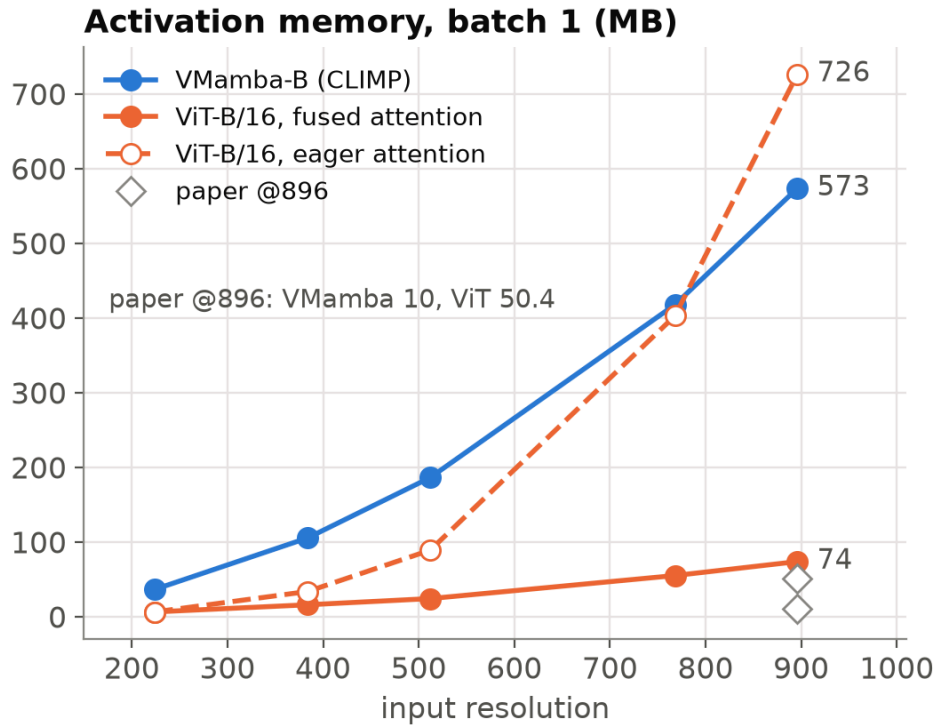
NoCaps @224	Align ↓	Unif. img ↓	Unif. txt ↓	Hub. img ↓	Hub. txt ↓
CLIMP (paper)	0.982	–3.18	–3.54	1.01	1.15
CLIMP (ours)	**0.982**	**–3.184**	**–3.546**	1.55	1.79
OpenCLIP (ours)	1.437	–2.724	–2.736	1.68	4.12

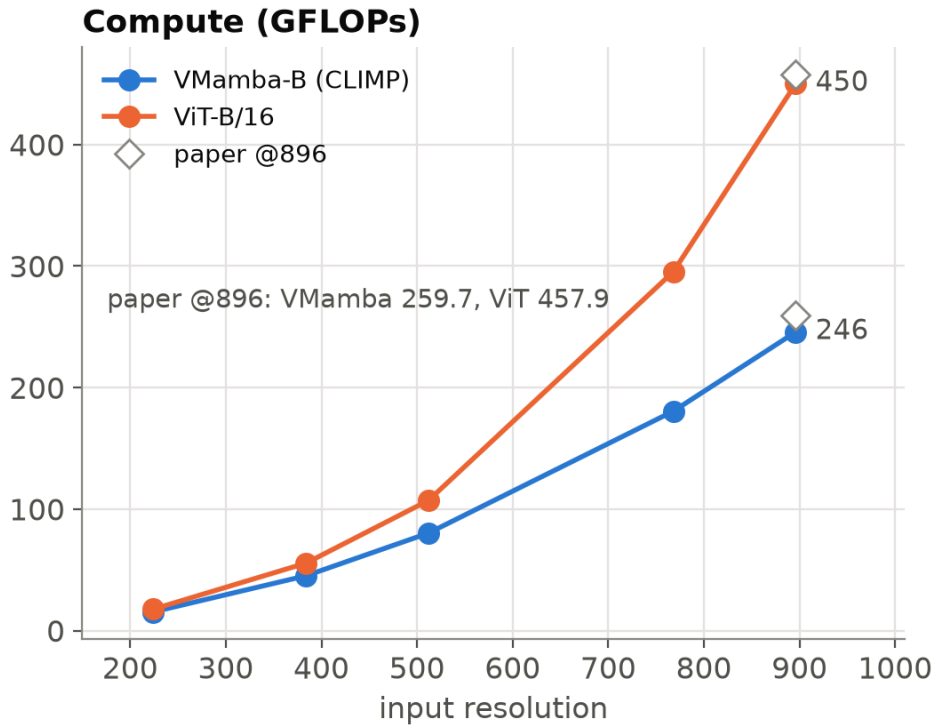
- REPRODUCED Alignment and uniformity match exactly. Hubness is higher; the paper never states its *k* (we use *k* = 10)
- CLIMP has **far better geometry** than OpenCLIP… yet **worse retrieval**. Geometry alone does not explain accuracy



NEW **CSLS** penalises "hub" items, which sit near everything. It needs no training and gives CLIMP **+9.3 pp** text R@1, twice OpenCLIP's gain. Transductive: uses the test set's own neighbourhoods.

Efficiency (E6, Fig. 3): FLOPs reproduce, the memory claim does not

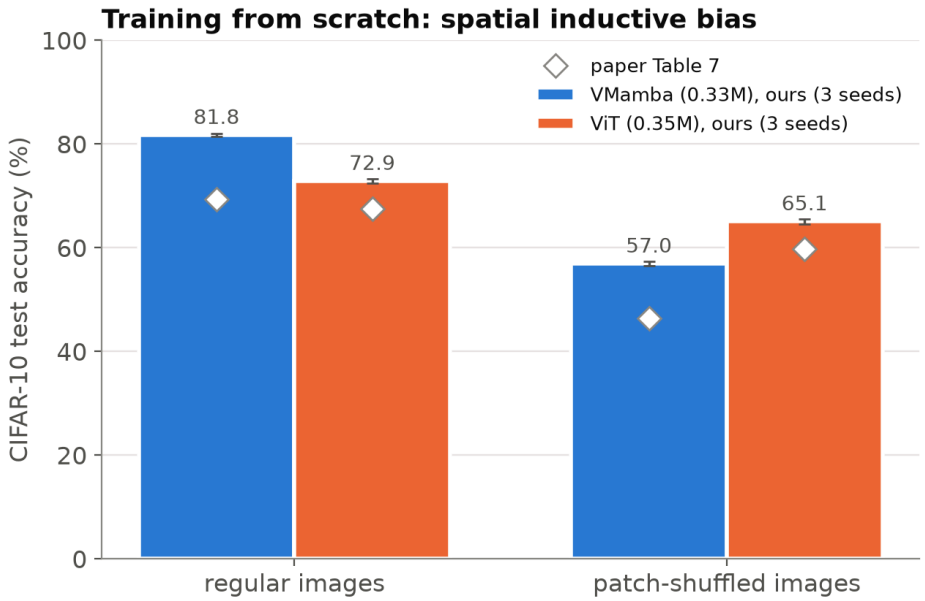


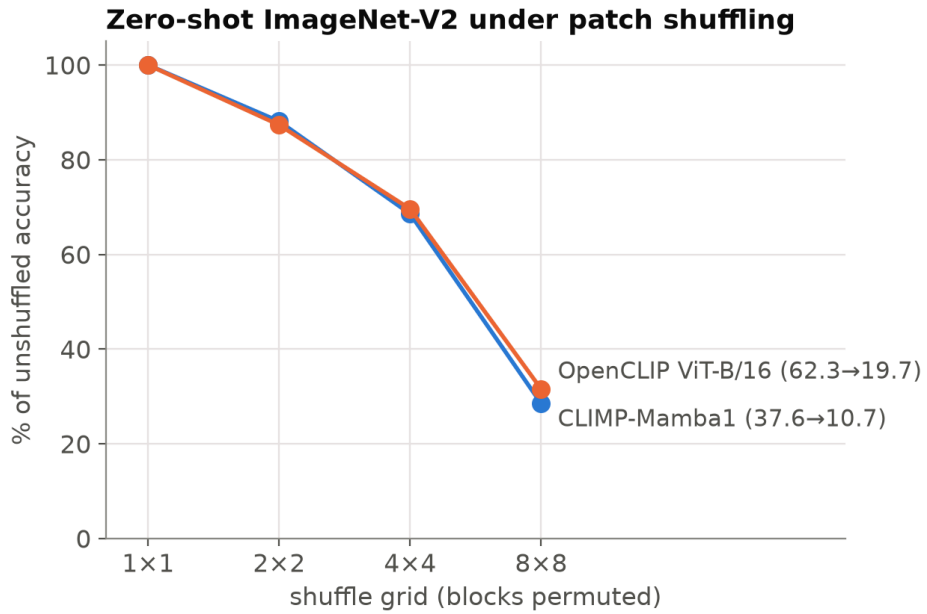


REPRODUCED FLOPs within 5% of the paper; VMamba needs **1.83× fewer** FLOPs at 896 px (paper: 1.8×). Quadratic attention shows up in ViT's curve.

NEW Memory depends on the attention kernel: VMamba uses **7.8× more** than a ViT with modern fused attention (573 vs 74 MB). It is lower only than "eager" attention, above ~768 px. The paper's 10 MB is never defined.

Spatial inductive bias (E8, Table 7) — and does it transfer?



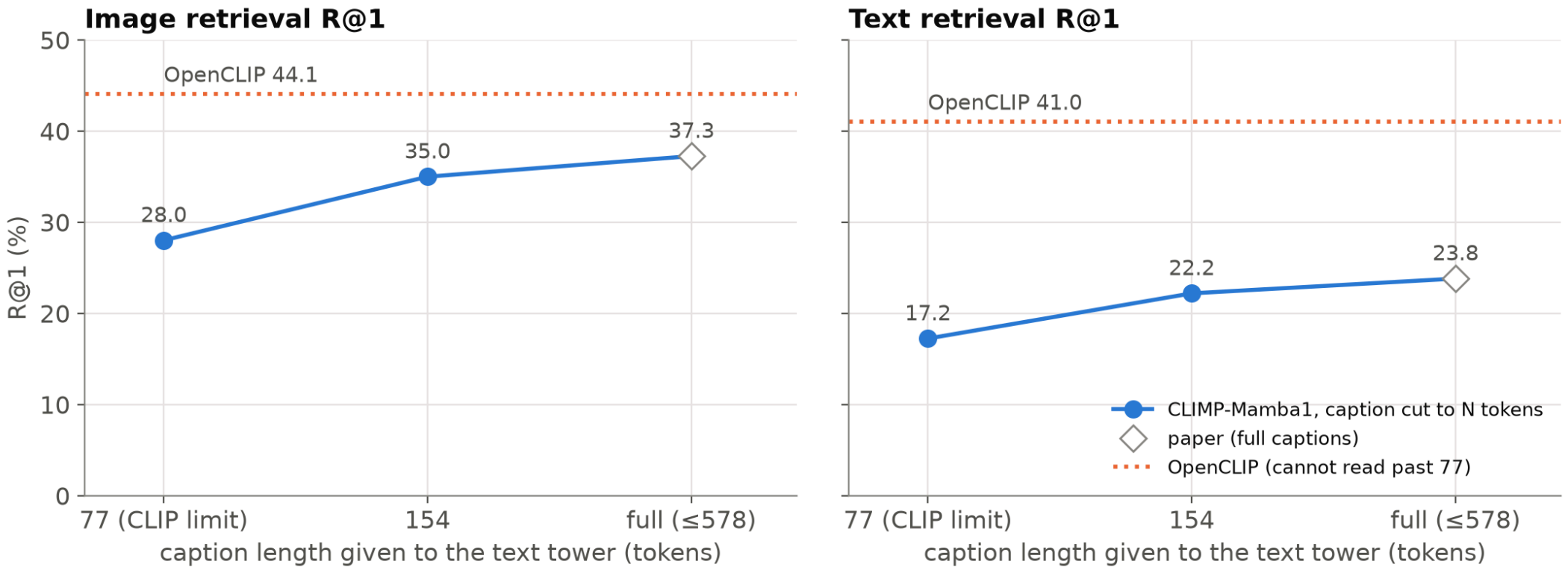


REPRODUCED Tiny models trained from scratch: **VMamba wins on normal images (+8.9 pp) and loses most when shuffled** (–24.8 vs –7.8 pp). Same pattern as the paper; higher absolute accuracy (the paper's recipe is unstated).

NEW On the **pretrained** CLIMP, shuffling blocks gives the **same relative drop** as the ViT at every grid size (–31% vs –30% at 4×4).

The scan-order bias is real when data is scarce, but at pretraining scale **it does not make CLIMP more layout-sensitive than a ViT**, so it cannot explain the robustness claims by itself.

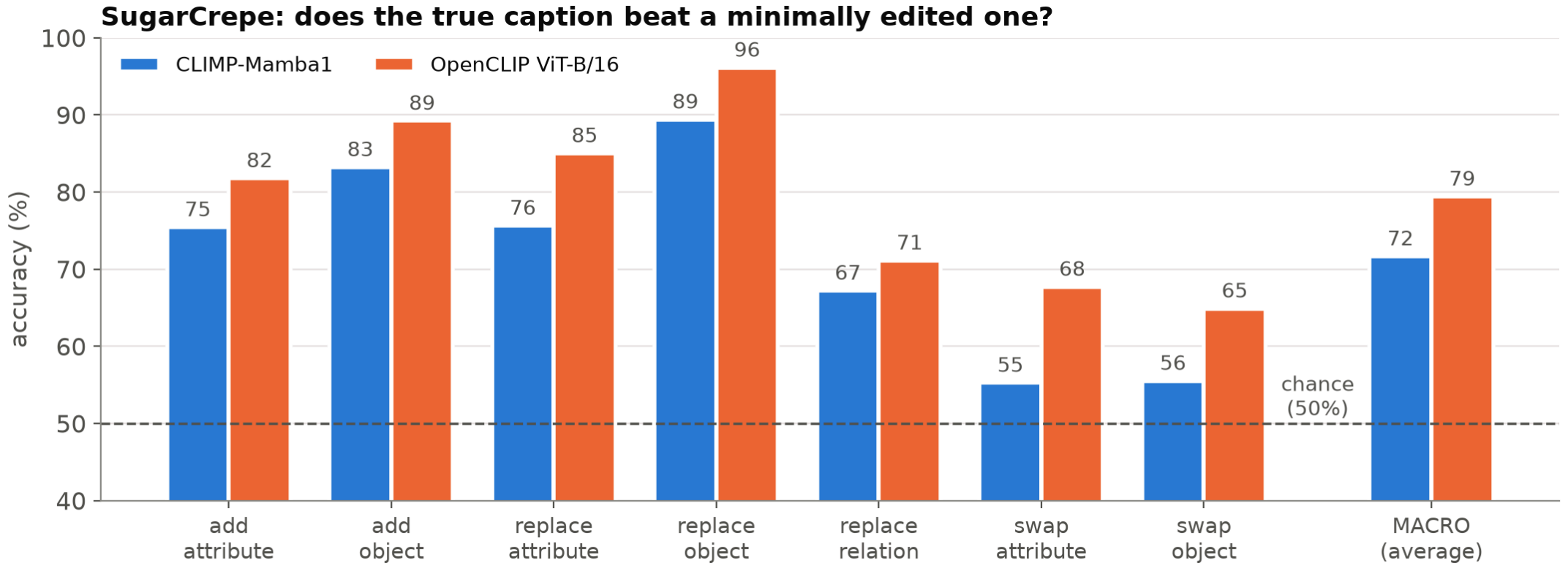
Long captions (E5, Table 6): reading more text helps — up to a point



REPRODUCED Full-caption DOCCI retrieval matches Table 6 (37.3 / 23.8 R@1). **NEW** Cutting captions to CLIP's 77 tokens costs CLIMP **9.2 pp** image R@1; **~75% of the gain arrives by 154 tokens**.

But OpenCLIP, which reads **less than half** of each caption, still wins on every metric (44.1 vs 37.3 image R@1, 41.0 vs 23.8 text R@1). Long context does not overcome a weaker base model.

Compositional understanding NEW (SugarCrepe, 7,511 items)



CLIMP trails OpenCLIP on every subset (macro **71.7 vs 79.4**), and is **near chance on "swap"**: "a white dog and a black cat" vs "a black dog and a white cat".

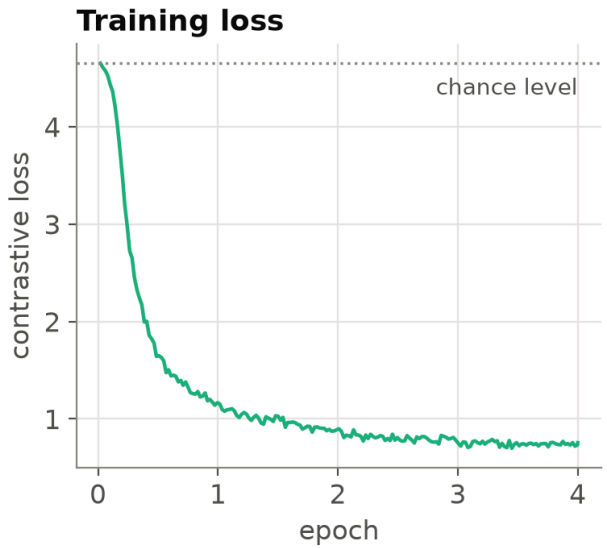
Surprising, because the text tower is a 1.4B **causal** LM that is sensitive to word order. The contrastive objective never forces it to use that, so CLIMP inherits CLIP's **"bag of words"** behaviour.

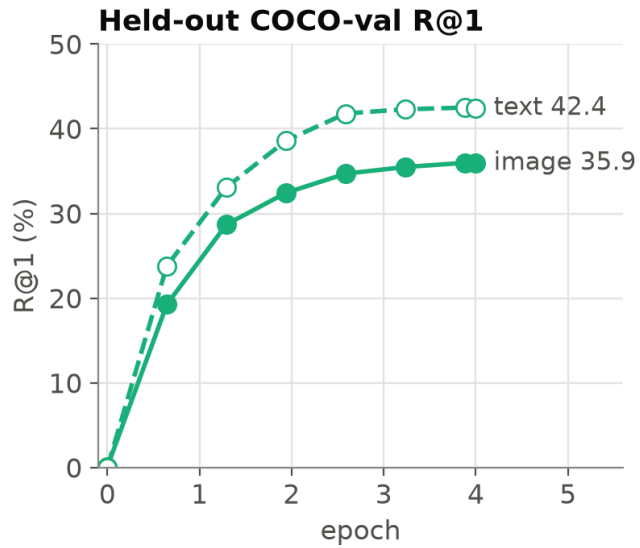
Retraining CLIMP ourselves, without the released weights NEW

Why: matching the released checkpoints validates the *evaluation*. Does the *recipe* work when we train it?

	Paper	Ours
Vision init	VMamba-B ImageNet-1K	**same** (official ckpt)
Text tower	Mamba-1.4B, **trained**	Mamba-1.4B, **frozen**
Loss / optimiser	InfoNCE, AdamW 5e-5 cosine	**same**
Data	CC12M (12M)	**COCO (118k)**, 4 epochs
Batch	2048	96 (2× T4, 4.7 h)

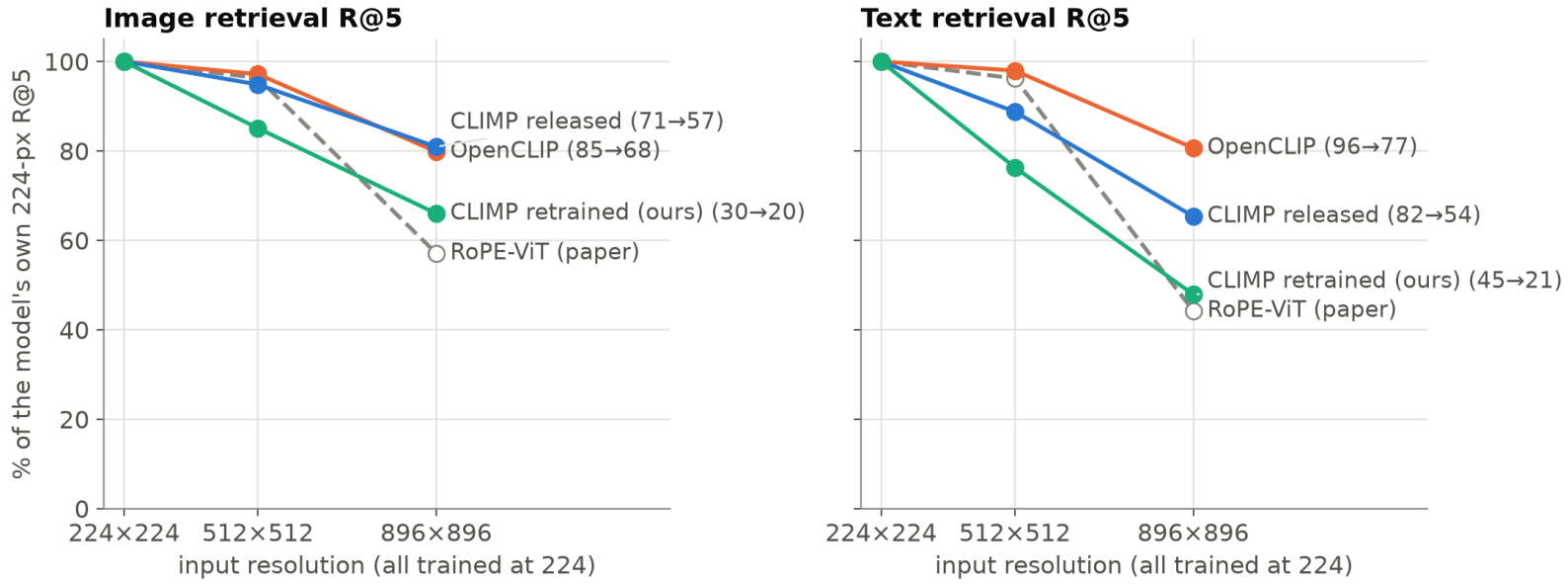
The frozen text tower is a forced deviation: 1.4B parameters don't fit on a T4 for training. I verified that the authors **did** train theirs (all 482 tensors differ from pretrained Mamba).





Training works: from chance to **36 / 42** R@1 on held-out COCO. Gains flatten after ~2.5 epochs.

Retrained CLIMP: what survives at small scale NEW



(top-1 / R@1)	Ours retrained	Released
ImageNet-V2 / **ImageNet-O**	13.9 / **22.5**	37.6 / **49.8**
DOCCI full captions @896 (img / txt)	**2.8 / 2.5**	37.3 / 23.8
Hubness image (↓)	**19.7**	1.55

- **No graceful high-res behaviour:** –34% / –52%, like the paper's RoPE-ViT. It comes from scale, not the architecture alone
- **ImageNet-O is still the best variant** (as released)
- **Long text needs a trained text tower**

Strengths and limitations

Strengths

- **Released artefacts are trustworthy:** 32/34 numbers matched within 0.5 pp
- **Clean, real mechanisms:**
  - any resolution with one set of weights (verified)
  - any caption length, and it helps (+9.2 pp)
  - 1.8× fewer FLOPs at 896 px
- **Controlled comparison:** same data and recipe isolate the architecture
- **A simple, convincing bias test** (CIFAR), reproduced over 3 seeds

Limitations

- **Weak, unreleased baselines.** Their RoPE-ViT reaches 34.9 text R@5 at 896; an off-the-shelf ViT gets 77.2
- **Headline rests on one dataset:** the "beats 167× data" claim is ImageNet-O only
- **Memory claim under-specified;** reversed against fused attention
- **Geometry → accuracy asserted, not shown;** hubness *k* unstated (text says 1.13, table says 1.15)
- **Small-scale bias ≠ pretrained behaviour** (patch-shuffle test)
- **Minor issues:**
  - Table 4's I→T / T→I labels appear swapped
  - CLIMP still drops 19–35% at 896 px



Is the evidence sufficient? Assumptions and improvements

Sufficient?

- For **"SSMs are a viable CLIP backbone at equal data"**: largely **yes**
- For **"better than Transformers"**: **no**. That would need tuned, released baselines, one larger-data comparison, memory measured with fused attention, and error bars on the big models

Assumptions the method makes

- **4 one-dimensional scans ≈ 2-D neighbourhoods.** Only partly: vertical neighbours are far apart in a row scan
- **The last token of a causal LM is a good sentence embedding.** It works, but word order goes unused (SugarCrepe)
- **Training at 224 px suffices for any resolution.** It runs, but accuracy still falls

What I would improve

1. **Release baselines;** define memory precisely (peak allocated, batch size, attention kernel)
2. **Hubness-aware inference** (CSLS / inverted softmax): free, +4.5 to +9.3 pp R@1
3. **Brief high-resolution fine-tuning** of the projection layers
4. **Compositional hard negatives** (NegCLIP-style) to use the LM's word-order sensitivity
5. **Test bias claims on the pretrained model** (my patch-shuffle protocol), not only on CIFAR



Impact, relation to LLMs, and conclusion

Where this fits

- Part of the shift to **sub-quadratic and hybrid models**: Mamba, Mamba-2, and hybrid Mamba–Transformer LLMs (Jamba, Nemotron)
- **Decoder LLMs as embedding models:** last-token pooling of a causal LM, as in LLM2CLIP and LLM-based text embedders
- **Vision encoders for multimodal LLMs:** documents and charts mean thousands of visual tokens. Linear compute and no position table are attractive, but memory needs a fair comparison
- **Long-context retrieval:** reading the whole description matters (+9.2 pp)
- Published 2026, so its long-term influence is not yet visible

Conclusion

- ✓ Released CLIMP reproduces: NoCaps at all resolutions, ImageNet variants, DOCCI, geometry, FLOPs, CIFAR bias
- ✗ Not robust to a broader test: the memory saving, hubness values, and advantages over a strong ViT (ImageNet-O excepted)
- NEW Long text helps (+9.2 pp) · CSLS +9.3 pp · pretrained layout sensitivity = ViT's · weak at word order
- NEW Retrained on COCO: it learns, but resolution robustness and long-text retrieval do **not** appear at small scale

> **Mamba is a viable CLIP backbone, but not yet a better one.**



Team Contribution

Student	Contribution	Score
Tanish Wanve	Equal (sole member)	**1.0**

Understanding and presentation: read the paper and the upstream code; report; slides; Q&A preparation

Experiments (8): NoCaps retrieval at 3 resolutions, embedding geometry, zero-shot ImageNet-V2/R/A/O, DOCCI long captions, memory/FLOPs, CIFAR inductive bias (3 seeds), plus 4 new analyses

Engineering:

- evaluation code with unit tests, verified against upstream
- correctness gate
- two-GPU Kaggle pipeline
- CLIMP retraining pipeline (distributed loss tested to equal the full batch)
- fixes for out-of-memory and dataset-loader failures
- every table and figure generated from result files

Single-member project, built with an AI coding assistant. All analysis, interpretation and choices are my own.

Appendix A — full NoCaps retrieval (E1)

Model	Res	Image R@1 / R@5 / R@10	Text R@1 / R@5 / R@10
CLIMP-Mamba1	224	**38.2 / 70.5 / 81.8** (README 38.2 / 70.5 / 81.8)	**51.8 / 81.9 / 90.9** (README 51.8 / 81.9 / 90.9)
CLIMP-Mamba1	512	34.2 / 66.9 / 79.2	42.2 / 72.7 / 83.8
CLIMP-Mamba1	896	26.2 / 57.1 / 70.8	25.3 / 53.5 / 66.5
CLIMP-Mamba2	224	37.4 / 70.2 / 81.7 (README 37.4 / 70.2 / 81.6)	51.0 / 81.2 / 90.3 (README 51.0 / 81.3 / 90.9)
CLIMP-Mamba2	896	25.8 / 57.1 / 70.8	24.5 / 53.9 / 66.6
OpenCLIP ViT-B/16	224	58.0 / 84.6 / 91.2	80.0 / 95.7 / 98.2
OpenCLIP ViT-B/16	512	54.3 / 82.2 / 89.6	74.4 / 93.8 / 97.2
OpenCLIP ViT-B/16	896	39.4 / 67.6 / 77.9	51.7 / 77.2 / 84.9

Image retrieval = text query → images; text retrieval = image query → captions (upstream definitions). Mamba-2 @512: 33.9 / 66.4 / 79.2 and 40.5 / 72.4 / 83.5.

Appendix B — implementation and deviations

Pipeline

- `src/exp_*.py` : one script per experiment; each writes a JSON with its config, library versions, GPU, git hashes, runtime and peak RAM
- `src/make_results.py` : builds every table and figure from those JSONs only. No hand-typed numbers
- Unit tests, including recall@k identical to upstream `eval_nocaps.py`
- **Smoke runs first** on tiny subsets; they caught an out-of-memory error on long DOCCI captions and a library conflict
- Upstream code pinned at commit `3ff1279` , never edited

Deviations (all in ASSUMPTIONS.md)

- Baseline: OpenCLIP ViT-B/16 (LAION-2B) instead of the unreleased RoPE-ViT / FlexViT / NaFlex + LLaMA
- Hubness $k = 10$ (unstated in the paper); CSLS is transductive
- E8: a fresh per-image patch permutation; 20 epochs (unstated in the paper)
- Datasets loaded from official parquet / jsonl mirrors (same rows and order)
- E3 uses ImageNet-V2/R/A/O; ImageNet-Sketch was not run